

Глава 10

Функциональные объекты STL и лямбда-функции

В главе подробно рассматриваются механизмы передачи функциональности алгоритмам и функциям-членам — *функциональные объекты* (function objects), или *функторы* (functors), введенные в разделе 6.10. В ней описано полное множество функциональных объектов и адаптеров функций, а также привязки (связыватели — binders) и концепция функциональной композиции, приведены примеры пользовательских функциональных объектов и детали использования лямбда-функций (упоминаемых в разделах 3.1.10 и 6.9).

В заключение читатели узнают об устройстве и удивительном поведении алгоритмов `remove_if()` и `for_each()`.

10.1. Концепция функциональных объектов

Функциональный объект, или *функтор*, — это объект, имеющий определенный оператор `()`, так что в примере

```
Тип_функтора fo;  
...  
fo(...);
```

выражение `fo()` — это вызов оператора `()` для функционального объекта `fo`, а не вызов функции `fo()`. Функциональный объект можно рассматривать как обычную функцию, записанную необычным способом. Вместо того чтобы записать все операторы в теле функции:

```
void fo() {  
    инструкции  
}
```

мы записываем их в теле оператора `()` в классе функционального объекта:

```
class FunctionObjectType {  
public:  
    void operator() () {  
        инструкции  
    }  
};
```

Несмотря на свою сложность, это определение имеет три важных преимущества.

1. Функциональный объект может быть более интеллектуальным, обладать большими возможностями, потому что у него есть состояние (ассоциированные члены, влияющие на поведение функционального объекта). Фактически вы можете иметь

два экземпляра одного и того же класса функционального объекта, которые могут одновременно иметь разные состояния. Обычные функции такими возможностями не обладают.

2. Каждый функциональный объект имеет свой тип. Следовательно, его тип можно передавать как шаблонный параметр, чтобы описать определенное поведение, и вы получите отличающиеся друг от друга контейнерные типы с разными функциональными объектами.
3. Функциональный объект обычно работает быстрее, чем указатель на функцию.

Более подробно эти преимущества объясняются в разделе 6.10.1, который содержит также пример, демонстрирующий, что функциональные объекты могут обладать более развитой логикой, чем обычные функции.

В следующих двух подразделах приводятся еще два примера, более подробно иллюстрирующие работу функциональных объектов. Первый пример показывает, как извлечь выгоду из того факта, что функциональный объект обычно имеет свой тип. Второй пример демонстрирует, как можно использовать состояние функционального объекта, и приводит к интересному свойству алгоритма `for_each()`, описанного в разделе 10.1.3.

10.1.1. Функциональные объекты как критерий сортировки

Программистам часто нужны упорядоченные коллекции элементов некоторого класса (например, коллекция объектов класса `Person`). Однако использовать для сортировки этих объектов оператор `<` иногда нежелательно, а иногда просто невозможно. Вместо этого желательно упорядочить эти объекты в соответствии с критерием сортировки, заданным некоторой функцией-членом. В этой ситуации может помочь функциональный объект. Рассмотрим следующий пример:

```
// fo/sort1.cpp

#include <iostream>
#include <string>
#include <set>
#include <algorithm>
using namespace std;

class Person {
public:
    string firstname() const;
    string lastname() const;
    ...
};

// класс функционального предиката
// - операция () возвращает результат проверки того, что
// первый объект класса Person меньше второго объекта
class PersonSortCriterion {
public:
    bool operator() (const Person& p1, const Person& p2) const {
        // объект класса Person меньше другого объекта класса Person
```

```

        // - если фамилия первого объекта меньше
        // - если фамилии совпадают, но имя первого объекта меньше
        return p1.lastname() < p2.lastname() ||
            (p1.lastname() == p2.lastname() &&
             p1.firstname() < p2.firstname());
    }
};
int main()
{
    // создаем множество со специальным критерием сортировки
    set<Person, PersonSortCriterion> coll;
    ...

    // что-то делаем с его элементами
    for (auto pos = coll.begin(); pos != coll.end(); ++pos) {
        ...
    }
    ...
}

```

Множество `coll` использует специальный критерий сортировки `PersonSortCriterion`, определенный как класс функционального объекта. В классе `PersonSortCriterion` оператор `()` определен так, что он сравнивает два объекта класса `Person` по их фамилиям, а если они совпадают, по их именам. Конструктор множества `coll` автоматически создает экземпляр класса `PersonSortCriterion`, так что элементы упорядочиваются именно по этому критерию сортировки.

Отметим, что критерий сортировки `PersonSortCriterion` является *типом*. Следовательно, его можно использовать как шаблонный параметр множества. Это было бы невозможно, если бы мы реализовали критерий сортировки с помощью обычной функции (как показано в разделе 6.8.2).

Все множества с таким критерием сортировки имеют свой собственный тип. Их нельзя комбинировать или присваивать множествам, имеющим “обычный” или другой пользовательский критерий сортировки. Следовательно, ни одна операция множества не может повлиять на автоматический критерий сортировки; однако можно определить функциональные объекты, задающие разные критерии сортировки с одним и тем же типом (см. раздел 7.8.6).

10.1.2. Функциональные объекты, имеющие внутреннее состояние

Следующий пример демонстрирует, как можно использовать функциональные объекты в качестве функций, одновременно имеющих несколько состояний:

```

// fo/sequence1.cpp

#include <iostream>
#include <list>
#include <algorithm>
#include <iterator>
#include "print.hpp"
using namespace std;

```

```

class IntSequence {
private:
    int value;
public:
    IntSequence (int initialValue) // конструктор
        : value(initialValue) {

    }

    int operator() () { // ''вызов функции''
        return value++;
    }
};

int main()
{
    list<int> coll;

    // вставляем значения от 1 до 9
    generate_n (back_inserter(coll), // старт
                9,                    // количество элементов
                IntSequence(1));      //генерирует значения, начиная с 1
    PRINT_ELEMENTS(coll);

    // заменяем элементы от второго до последнего
    // последовательностью элементов, начинающуюся с 42
    generate (next(coll.begin()), //старт
              prev(coll.end()),   // конец
              IntSequence(42));   //генерирует значения, начиная с 42

    PRINT_ELEMENTS(coll);
}

```

В этом примере функциональный объект `IntSequence` генерирует последовательность целых чисел. При каждом вызове оператор `()` возвращает текущее значение и выполняет его инкремент. Начальное значение можно задать с помощью аргумента конструктора.

Затем два таких функциональных объекта используются алгоритмами `generate()` и `generate_n()`, которые записывают сгенерированные значения в коллекцию: выражение `IntSequence(1)` в вызове

```

generate_n (back_inserter(coll),
            9,
            IntSequence(1));

```

создает такой функциональный объект, инициализированный значением 1. Алгоритм `generate_n()` использует его девять раз для записи элемента, генерируя значения от 1 до 9. Аналогично выражение `IntSequence(42)` генерирует последовательность чисел, начинающуюся с 42. Алгоритм `generate()` заменяет элементы со второго и до последнего¹. Результат работы программы выглядит следующим образом:

¹ Функции `std::next()` и `std::prev()` предусмотрены стандартом C++11 (см. раздел 9.3.2). Отметим, что выражения `++coll.begin()` и `--coll.end()` могут иногда не компилироваться (см. раздел 9.2.6)..


```
1 2 3 4 5 6 7 8 9
1 42 43 44 45 46 47 48 9
```

Используя другие варианты оператора `()`, можно легко создавать более сложные последовательности.

По умолчанию функциональные объекты передаются по значению, а не по ссылке. Следовательно, этот алгоритм не изменяет состояния функционального объекта. Например, следующий код дважды генерирует последовательность, начинающуюся с 1:

```
IntSequence seq(1); // последовательность целых чисел, начинающаяся с 1

// вставляем последовательность, начинающуюся с 1
generate_n (back_inserter(coll), 9, seq);

// снова вставляем последовательность, начинающуюся с 1
generate_n (back_inserter(coll), 9, seq);
```

Передача функциональных объектов по значению, а не по ссылке позволяет передавать константные и временные выражения. В противном случае передача выражения `IntSequence(1)` была бы невозможной.

Недостатком передачи функционального объекта по значению является то, что мы не можем извлечь выгоду из модификаций состояния функциональных объектов. Алгоритмы могут модифицировать состояние функционального объекта, но получить доступ и обработать заключительное состояние невозможно, поскольку они создают внутренние копии функциональных объектов. В то же время доступ к заключительному состоянию может быть необходим, поэтому возникает вопрос, как получить “результат” работы алгоритма.

Существуют три способа получить результат или обратную связь между функциональными объектами и алгоритмами.

1. Можно хранить состояние во внешней памяти и позволить функциональному объекту обращаться к нему.
2. Можно передать функциональный объект по ссылке.
3. Можно использовать значение, возвращаемое алгоритмом `for_each()`.

Последняя возможность обсуждается в следующем подразделе.

Для передачи функционального объекта по ссылке нужно просто указать, что при вызове алгоритма типом функционального объекта является ссылка². Рассмотрим пример:

```
// fo/sequence2.cpp

#include <iostream>
#include <list>
#include <algorithm>
#include <iterator>
#include "print.hpp"
using namespace std;

class IntSequence {
private:
    int value;
```

² Благодаря за это замечание Филипа Кёстера (Philip Köster)

```

public:
    // конструктор
    IntSequence (int initialValue)
        : value(initialValue) {
    }

    // "вызов функции"
    int operator() () {
        return value++;
    }
};

int main()
{
    list<int> coll;

    IntSequence seq(1); // последовательность целых чисел,
                        // начинающаяся с 1

    // вставляем элементы от 1 до 4
    // - передаем функциональный объект по ссылке
    // чтобы он продолжил работу, начиная с 5
    generate_n<back_inserter<list<int>>>,
               int, IntSequence&>(back_inserter(coll), // начало
                                   4,                    // количество элементов
                                   seq);                 // генерирует значения
    PRINT_ELEMENTS(coll);

    // вставляем значения от 42 до 45
    generate_n (back_inserter(coll), // начало
               4,                    // количество элементов
               IntSequence(42));     // генерирует значения
    PRINT_ELEMENTS(coll);

    // продолжаем первую последовательность
    // - передаем функциональный объект по значению
    // чтобы он снова начал работу с 5
    generate_n (back_inserter(coll), // начало
               4,                    // количество элементов
               seq);                 // генерирует значения
    PRINT_ELEMENTS(coll);

    // продолжаем первую последовательность
    generate_n (back_inserter(coll), // начало
               4,                    // количество элементов
               seq);                 // генерирует значения
    PRINT_ELEMENTS(coll);
}

```

Программа выводит на экран следующие результаты:

```

1 2 3 4
1 2 3 4 42 43 44 45

```

```
1 2 3 4 42 43 44 45 5 6 7 8
1 2 3 4 42 43 44 45 5 6 7 8 5 6 7 8
```

При первом вызове алгоритма `generate_n()` функциональный объект `seq` передается по ссылке. Для этого явно указывается шаблонный параметр.

```
generate_n<back_inserter_iterator<list<int>>>,
          int, IntSequence&>(back_inserter(coll), // начало
                             4,                  // количество элементов
                             seq);               // генерирует значения
```

В результате внутреннее значение функционального объекта `seq` после вызова изменяется, и при втором использовании объекта `seq` (при третьем вызове) алгоритм `generate_n()` продолжает работу с последовательностью, полученной после первого вызова. Однако в этом вызове функциональный объект `seq` передается по значению.

```
generate_n (back_inserter(coll), // начало
            4,                  // количество элементов
            seq);               // количество значений
```

Следовательно, этот вызов не изменяет состояние функционального объекта `seq`. В результате последний вызов алгоритма `generate_n()` продолжает последовательность снова со значения 5.

10.1.3. Значение, возвращаемое алгоритмом `for_each()`

Передавать функциональный объект по ссылке, чтобы получить доступ к его заключительному состоянию, не обязательно, если использовать алгоритм `for_each()`. Этот алгоритм имеет уникальную возможность возвращать свой функциональный объект (ни один другой алгоритм делать это не может). Следовательно, можно запросить состояние вашего функционального объекта, проверив значение, возвращаемое алгоритмом `for_each()`.

Следующая программа представляет собой прекрасный пример использования значения, возвращаемого алгоритмом `for_each()`. Она демонстрирует процесс вычисления среднего значения последовательности:

```
// fo/foreach3.cpp

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

// функциональный объект для вычисления среднего значения
class MeanValue {
private:
    long num; // количество элементов
    long sum; // сумма всех значений
public:
    // конструктор
    MeanValue () : num(0), sum(0) {
    }
}
```

```

// "вызов функции"
// - обработка нескольких элементов последовательности
void operator() (int elem) {
    ++num;          // инкремент счетчика
    sum += elem;    // добавляем значение
}

// возвращаем среднее значение
double value () {
    return static_cast<double>(sum) / static_cast<double>(num);
}
};

int main()
{
    vector<int> coll = { 1, 2, 3, 4, 5, 6, 7, 8 };

    // вычисляем и выводим на экран среднее значение
    MeanValue mv = for_each (coll.begin(), coll.end(), // диапазон
                             MeanValue());           // операция
    cout << "mean value: " << mv.value() << endl;
}

```

Выражение `MeanValue()` создает функциональный объект, подсчитывающий количество элементов и вычисляющий сумму всех значений. При передаче этого функционального объекта в алгоритм `for_each()` он применяется к каждому элементу контейнера `coll`.

```

MeanValue mv = for_each (coll.begin(), coll.end(),
                         MeanValue());

```

Этот функциональный объект затем возвращается и присваивается объекту `mv`, поэтому мы можем запросить его состояние после выполнения этого оператора, вызвав функцию `mv.value()`. Таким образом, эта программа выводит на экран следующее значение:

```
mean value: 4.5
```

Класс `MeanValue` можно настроить более тонко, определив автоматическое преобразование типа в `double`. Затем можно непосредственно использовать среднее значение, вычисленное алгоритмом `for_each()`. Пример приведен в разделе 11.4.

Следует заметить, что лямбда-функции позволяют более удобно описать это поведение (соответствующий пример приведен в разделе 10.3.). Однако это не означает, что лямбда-функции всегда лучше функциональных объектов. Функциональные объекты являются более удобными в тех ситуациях, в которых требуется знать их тип, например, в объявлении хеш-функций, критерия сортировки, критерия эквивалентности или в неупорядоченных контейнерах. Тот факт, что функциональные объекты, как правило, вводятся глобально, позволяет включать их в заголовочные файлы и библиотеки, а лямбда-функции целесообразнее использовать для описания специфичного поведения, заданного локально.

10.1.4. Предикаты и функциональные объекты

Предикаты — это функции или функциональные объекты, возвращающие булево значение (значение, допускающее преобразование в тип `bool`). Однако не всякая функция,

возвращающая булево значение, является корректным предикатом для библиотеки STL. Это может привести к недоразумениям. Рассмотрим следующий пример:

```
// fo/removeif1.cpp

#include <iostream>
#include <list>
#include <algorithm>
#include "print.hpp"
using namespace std;

class Nth { // функциональный объект, возвращающий
           // значение true при n-м вызове
private:
    int nth;    // вызов, для которого возвращается
               // значение true
    int count; // счетчик вызовов
public:
    Nth (int n) : nth(n), count(0) {
    }
    bool operator() (int) {
        return ++count == nth;
    }
};

int main()
{
    list<int> coll = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
    PRINT_ELEMENTS(coll, "coll: ");

    // удаляем третий элемент
    list<int>::iterator pos;
    pos = remove_if(coll.begin(), coll.end(), // диапазон
                   Nth(3));                 // критерий удаления
    coll.erase(pos, coll.end());
    PRINT_ELEMENTS(coll, "3rd removed: ");
}
```

В этой программе определяется функциональный объект `Nth`, возвращающий значение `true` при n -м вызове. Однако, если передать его алгоритму `remove_if()`, результат работы алгоритма, удаляющего все элементы, для которого унарный предикат возвращает значение `true` (см. раздел 11.7.1), окажется весьма странным³.

```
coll: 1 2 3 4 5 6 7 8 9 10
3rd removed: 1 2 4 5 7 8 9 10
```

Удалены два элемента — третий и шестой. Это произошло потому, что обычная реализация алгоритма автоматически копирует предикат во время работы алгоритма.

```
template <typename ForwIter, typename Predicate>
ForwIter std::remove_if(ForwIter beg, ForwIter end,
```

³ По крайней мере так получается при работе с компиляторами `gcc 4.5` и `Visual C++ 2010`, но другие платформы могут привести к другим результатам; этот вопрос обсуждается ниже.

```

        Predicate op)
{
    beg = find_if(beg, end, op);
    if (beg == end) {
        return beg;
    }
    else {
        ForwIter next = beg;
        return remove_copy_if(++next, end, beg, op);
    }
}

```

Этот алгоритм использует алгоритм `find_if()` для поиска первого элемента, подлежащего удалению. Однако затем для обработки оставшихся элементов, если они есть, алгоритм использует копию переданного предиката `op`. В этом месте функциональный объект `Nth` используется в его исходном состоянии, и алгоритм снова удаляет третий среди оставшихся элементов, который фактически является седьмым.

Это — не ошибка. Стандарт не регламентирует, как часто предикат может копироваться в алгоритме. Следовательно, для того чтобы поведение стандартной библиотеки C++ было гарантированным, не следует передавать функциональный объект, поведение которого зависит от того, сколько раз он копируется или вызывается. Таким образом, если вызывать унарный предикат два раза с двумя одинаковыми аргументами, то оба раза предикат обязан вернуть один и тот же результат.

Другими словами: **предикат не должен иметь состояния**. Иначе говоря, предикат не должен изменять свое состояние вследствие вызова, а копия предиката должна иметь такое же состояние, что и оригинал. Для того чтобы запретить изменение состояния предиката во время вызова функции, оператор `()` необходимо определить как константную функцию-член.

Описанное выше неожиданное поведение можно предотвратить и гарантировать, что алгоритм будет работать как надо, даже с таким функциональным объектом, как `Nth`, не снижая быстродействия. Можно реализовать алгоритм `remove_if()` так, чтобы вызов алгоритма `find_if()` заменялся его содержимым.

```

template <typename ForwIter, typename Predicate>
ForwIter std::remove_if(ForwIter beg, ForwIter end,
                       Predicate op)
{
    while (beg != end && !op(*beg)) {
        ++beg;
    }

    if (beg == end) {
        return beg;
    }
    else {
        ForwIter next = beg;
        return remove_copy_if(++next, end, beg, op);
    }
}

```

Итак, было бы целесообразно заменить реализацию алгоритма `remove_if()` или сообщить разработчикам библиотеки о необходимости сделать это. Насколько я знаю, эта проблема в действующей реализации библиотеки возникла только при работе с алгоритмом `remove_if()`. Алгоритм `remove_copy_if()` работает так, как ожидается. (Вопрос, следует ли стандартной библиотеке C++ гарантировать ожидаемое поведение в таких ситуациях, как в предыдущем примере, все еще остается предметом обсуждений.) Однако для обеспечения переносимости программ никогда не следует полагаться на детали реализации. Оператор вызова предиката всегда следует объявлять как константную функцию-член.

При работе с лямбда-функциями состояние можно сделать общим для всех копий функционального объекта, так что эта проблема не возникает. Подробности описаны в разделе 10.3.2.

10.2. Стандартные функциональные объекты и привязки

Как указано в разделе 6.10.2, стандартная библиотека C++ содержит много стандартных функциональных объектов и привязок, позволяющих создавать более сложные функциональные объекты.

Эта возможность называется *функциональной композицией* (functional composition). Она основана на фундаментальных функциональных объектах и адаптерах, описанных в этом разделе. Для использования этих функциональных объектов и привязок в программе следует включить заголовочный файл `<functional>`.

```
#include <functional>
```

10.2.1. Стандартные функциональные объекты

Все предопределенные функциональные объекты перечислены в табл. 10.1 (функциональные объекты `bit_and`, `bit_or` и `bit_xor` предусмотрены стандартом C++11).

Таблица 10.1. Стандартные функциональные объекты

Выражение	Результат
<code>negate<type>()</code>	$- param$
<code>plus<type>()</code>	$param1 + param2$
<code>minus<type>()</code>	$param1 - param2$
<code>multiplies<type>()</code>	$param1 * param2$
<code>divides<type>()</code>	$param1 / param2$
<code>modulus<type>()</code>	$param1 \% param2$
<code>equal_to<type>()</code>	$param1 == param2$
<code>not_equal_to<type>()</code>	$param1 != param2$
<code>less<type>()</code>	$param1 < param2$
<code>greater<type>()</code>	$param1 > param2$

Окончание табл. 10.1

Выражение	Результат
<code>less_equal<type>()</code>	<code>param1 <= param2</code>
<code>greater_equal<type>()</code>	<code>param1 >= param2</code>
<code>logical_not<type>()</code>	<code>! param</code>
<code>logical_and<type>()</code>	<code>param1 && param2</code>
<code>logical_or<type>()</code>	<code>param1 param2</code>
<code>bit_and<type>()</code>	<code>param1 & param2</code>
<code>bit_or<type>()</code>	<code>param1 param2</code>
<code>bit_xor<type>()</code>	<code>param1 ^ param2</code>

Функциональный объект `less<>` — это критерий, заданный по умолчанию и предназначенный для упорядочения и сравнения объектов с помощью функций сортировки и ассоциативных контейнеров. Таким образом, операции сортировки, заданные по умолчанию, всегда создают возрастающий порядок (*element < nextElement*). Функциональный объект `equal_to<>` — это критерий эквивалентности по умолчанию для неупорядоченных контейнеров.

Для сравнения строк на разных языках стандартная библиотека C++ позволяет задавать функциональные объекты локализации в качестве критерия сортировки строк (см. раздел 16.3).

10.2.2. Функциональные адаптеры и привязки

Функциональный адаптер — это функциональный объект, предназначенный для создания композиций, состоящих из функциональных объектов, определенных значений и специальных функций (соответствует шаблону *Компоновщик* в книге [GoF:DesignPatterns]). Однако со временем способ создания композиций из функциональных объектов изменился. Фактически все такие свойства, предусмотренные в стандарте C++98, объявлены устаревшими в стандарте C++11, в котором описаны более удобные и гибкие адаптеры. Сначала будет представлен современный способ создания композиций из функциональных объектов, а в разделе 10.2.4 будет дан более короткий обзор устаревших свойств.

В табл. 10.2 перечислены функциональные адаптеры, предусмотренные в стандартной библиотеке C++ в соответствии со стандартом C++11.

Таблица 10.2. Стандартные функциональные адаптеры

Выражение	Результат
<code>bind(op, args...)</code>	Связывает список аргументов <i>args</i> с операцией <i>op</i>
<code>mem_fn(op)</code>	Вызывает операцию <i>op()</i> как функцию-член объекта или объекта, на который ссылается указатель
<code>not1(op)</code>	Унарное отрицание: <code>!op(param)</code>
<code>not2(op)</code>	Бинарное отрицание: <code>!op(param1,param2)</code>

Наиболее важным адаптером является `bind()`, он позволяет делать следующее:

- адаптировать и создавать новые функциональные объекты из существующих или стандартных функциональных объектов;
- вызывать глобальные функции;
- вызывать функции-члены объектов, указателей на объекты и интеллектуальных указателей на объекты.

Адаптер `bind()`

В принципе адаптер `bind()` связывает параметры *вызываемых объектов* (см. раздел 4.4). Следовательно, если функция, функция-член, функциональный объект или лямбда-функция требует каких-то параметров, их можно связать с конкретными или передаваемыми аргументами. Конкретные аргументы можно просто назвать. Для передаваемых аргументов можно использовать стандартные *заполнители* `_1`, `_2`, ..., определенные в пространстве имен `std::placeholders`.

Как правило, привязки используются для указания параметров при использовании стандартных функциональных объектов, предусмотренных в стандартной библиотеке C++ (см. раздел 10.2.1). Рассмотрим пример:

```
// fo/bind1.cpp

#include <functional>
#include <iostream>

int main()
{
    auto plus10 = std::bind(std::plus<int>(),
                           std::placeholders::_1,
                           10);
    std::cout << "+10: " << plus10(7) << std::endl;

    auto plus10times2 = std::bind(std::multiplies<int>(),
                                  std::bind(std::plus<int>(),
                                              std::placeholders::_1,
                                              10),
                                  2);
    std::cout << "+10 *2: " << plus10times2(7) << std::endl;

    auto pow3 = std::bind(std::multiplies<int>(),
                          std::bind(std::multiplies<int>(),
                                      std::placeholders::_1,
                                      std::placeholders::_1),
                          std::placeholders::_1);
    std::cout << "x*x*x: " << pow3(7) << std::endl;

    auto inversDivide = std::bind(std::divides<double>(),
                                  std::placeholders::_2,
                                  std::placeholders::_1);
    std::cout << "invdiv: " << inversDivide(49,7) << std::endl;
}
```

Здесь определены четыре разных привязки, представляющие собой функциональные объекты. Например, привязка `plus10`, определенная как

```
std::bind(std::plus<int>(),
          std::placeholders::_1,
          10)
```

представляет собой функциональный объект, который автоматически вызывает функциональный объект `plus<>` (т.е. операцию `+`), передавая ему заполнитель `_1` в качестве первого параметра/операнда и `10` в качестве второго параметра/операнда. Заполнитель `_1` представляет первый аргумент, передаваемый выражению, как единое целое. Следовательно, для любого аргумента, передаваемого в это выражение, данный функциональный объект будет возвращать значение этого аргумента, увеличенное на `10`.

Для того чтобы избежать утомительного повторения пространства имен `placeholders`, можно использовать соответствующую директиву `using`. Таким образом, две директивы `using` позволяют сделать оператор более компактным.

```
using namespace std;
using namespace std::placeholders;
bind (plus<int>(), _1, 10) // param1+10
```

Привязку также можно вызывать непосредственно. Например, выражение

```
std::cout << std::bind(std::plus<int>(), _1, 10) (32) << std::endl;
```

запишет `42` в стандартный поток вывода, если передать этот функциональный объект алгоритму, и алгоритм может применить его к каждому элементу, с которым он работает. Рассмотрим пример:

```
// добавляем 10 к каждому элементу
std::transform (coll.begin(), coll.end(),           // источник
                coll.begin(),                       // получатель
                std::bind(std::plus<int>(), _1, 10)); // операция
```

Аналогично можно определить привязку, представляющую критерий сортировки. Например, для поиска первого элемента, превышающего `42`, можно связать функциональный объект `greater<>`, переданный в качестве первого аргумента, с числом `42`, указанным в качестве второго аргумента.

```
// ищем первый элемент > 42
auto pos = std::find_if (coll.begin(), coll.end(),
                        std::bind(std::greater<int>(), _1, 42))
```

Отметим, что для используемого стандартного функционального объекта всегда необходимо задавать тип аргумента. Если типы не совпадают, выполняется преобразование или возникает ошибка компиляции.

Остальные инструкции в этой программе показывают, что привязки могут быть вложенными и создавать еще более сложные функциональные объекты. Например, следующее выражение определяет функциональный объект, добавляющий число `10` к передаваемому аргументу, а затем умножающий его на `2` (пространство имен опущено).

```
bind(multiplies<int>(), // (param1+10)*2
     bind(plus<int>(), _1,
```

```
10),
2);
```

Как видим, выражения вычисляются в направлении изнутри наружу.

Аналогично можно возвести значение в куб, объединив два объекта `multiplies<>` с тремя заполнителями передаваемого аргумента.

```
bind(multiplies<int>(), // (param1*param1)*param1
    bind(multiplies<int>(), _1,
        _1),
    _1);
```

Последнее выражение определяет функциональный объект, в котором аргументы для деления переставлены местами. Таким образом, он делит второй аргумент на первый.

```
bind(divides<double>(), _2, // param2/param1
    _1);
```

Итак, программа выводит на экран следующие результаты:

```
+10: 17
+10 *2: 34
x*x*x: 343
invdiv: 0.142857
```

Другие примеры использования привязок приведены в разделе 6.10.3. В разделе 10.3.1 показано, как то же самое можно сделать с помощью лямбда-функций.

Вызов глобальных функций

Следующий пример демонстрирует, как можно использовать привязку `bind()` для вызова глобальных функций (вариант с использованием лямбда-функций приведен в разделе 10.3.3).

```
// fo/compose3.cpp

#include <iostream>
#include <algorithm>
#include <functional>
#include <locale>
#include <string>
using namespace std;
using namespace std::placeholders;

char myToupper (char c)
{
    std::locale loc;
    return std::use_facet<std::ctype<char> >(loc).toupper(c);
}

int main()
{
    string s("Internationalization");
    string sub("Nation");
```

```
// поиск подстроки без учета регистра
string::iterator pos;
pos = search (s.begin(),s.end(),      // строка, в которой идет поиск
             sub.begin(),sub.end(),   // искомая подстрока
             bind(equal_to<char>()),  // критерий сравнения
             bind(myToupper,_1),
             bind(myToupper,_2)));
if (pos != s.end()) {
    cout << "\"" << sub << "\" is part of \"" << s << "\""
        << endl;
}
}
```

Здесь алгоритм `search()` использован для проверки, является ли строка `sub` подстрокой строки `s` без учета регистра. С помощью привязки

```
bind(equal_to<char>()),
    bind(myToupper,_1),
    bind(myToupper,_2)));
```

создается вызов функционального объекта:

```
myToupper (param1)==myToupper (param2)
```

в котором `myToupper()` — пользовательская вспомогательная функция для преобразования строки в верхний регистр (см. раздел 16.4.4).

Отметим, что привязка `bind()` автоматически копирует передаваемые аргументы. Для того чтобы функциональный объект использовал ссылки на передаваемый аргумент, следует использовать функции `ref()` или `cref()` (см. раздел 5.4.3). Рассмотрим пример:

```
void incr (int& i)
{
    ++i;
}

int i=0;
bind(incr,i)();      // инкремент копии i, на i не влияет
bind(incr,ref(i))(); // инкремент i
```

Вызов функций-членов

Следующая программа демонстрирует, как привязка `bind()` позволяет вызвать функцию-член (вариант с использованием лямбда-функций приведен в разделе 10.3.3).

```
// fo/bind2.cpp

#include <functional>
#include <algorithm>
#include <vector>
#include <iostream>
#include <string>
```

```

using namespace std;
using namespace std::placeholders;

class Person {
private:
    string name;
public:
    Person (const string& n) : name(n) {
    }
    void print () const {
        cout << name << endl;
    }

    void print2 (const string& prefix) const {
        cout << prefix << name << endl;
    }
    ...
};

int main()
{
    vector<Person> coll
        = { Person("Tick"), Person("Trick"), Person("Track") };

    // вызываем функцию-член print() каждого объекта класса Person
    for_each (coll.begin(), coll.end(),
        bind(&Person::print, _1));
    cout << endl;

    // вызываем функцию-член print2() каждого объекта класса Person
    // с дополнительным аргументом
    for_each (coll.begin(), coll.end(),
        bind(&Person::print2, _1, "Person: "));
    cout << endl;

    // вызываем функцию-член print2() временного объекта класса Person
    bind(&Person::print2, _1, "This is: ") (Person("nico"));
}

```

Здесь привязка

```
bind(&Person::print, _1)
```

определяет функциональный объект, вызывающий функцию-член *param1*.print() переданного объекта класса Person. Иначе говоря, поскольку первый аргумент является функцией-членом, второй аргумент определяет объект, для которого эта функция вызывается.

Этой функции-члену передаются все дополнительные аргументы. Это значит, что привязка

```
bind(&Person::print2, _1, "Person: ")
```

определяет функциональный объект, вызывающий функцию-член *param1*.print2("Person: ") для любого переданного объекта класса Person.

Здесь передаваемые объекты являются членами вектора `coll`, но в принципе объекты можно передавать непосредственно. Например:

```
Person n("nico");
bind(&Person::print2, _1, "This is: ") (n);
```

вызывает `n.print2("This is: ")`.

Результаты работы этой программы имеют следующий вид:

```
Tick
Trick
Track
Person: Tick
Person: Trick
Person: Track
This is: nico
```

Отметим, что привязке `bind()` можно передавать также указатели на объекты и даже интеллектуальные указатели.

```
std::vector<Person*> cp;
...
std::for_each (cp.begin(), cp.end(),
               std::bind(&Person::print,
                        std::placeholders::_1));

std::vector<std::shared_ptr<Person>> sp;
...
std::for_each (sp.begin(), sp.end(),
               std::bind(&Person::print,
                        std::placeholders::_1));
```

Отметим, что с помощью привязки можно также вызывать модифицирующие функции-члены.

```
class Person {
public:
    ...
    void setName (const std::string& n) {
        name = n;
    }
};

std::vector<Person> coll;
...
std::for_each (coll.begin(), coll.end(), // присваиваем всем одинаковые имена
               std::bind(&Person::setName,
                        std::placeholders::_1,
                        "Paul"));
```

Возможен также вызов виртуальных функций-членов. Если связывается метод базового класса, а объект принадлежит производному классу, то будет вызвана правильная виртуальная функция производного класса.

Адаптер `mem_fn()`

Для вызова функций-членов можно также использовать адаптер `mem_fn()`, в котором можно не указывать заполнитель для объекта, для которого вызывается функция-член.

```
std::for_each (coll.begin(), coll.end(),
              std::mem_fn(&Person::print));
```

Таким образом, оператор `()` из объекта, возвращенного адаптером `mem_fn()`, просто вызывает инициализированную им функцию-член. Вызов функции относится к объекту, который передается как первый аргумент, а дополнительные аргументы передаются функции-члену в качестве параметров.

```
std::mem_fn(&Person::print) (n);           // вызывает n.print()

std::mem_fn(&Person::print2) (n, "Person: "); // вызывает n.print2("Person: ")
```

Однако для связывания дополнительного аргумента с функциональным объектом снова необходимо использовать привязку `bind()`.

```
std::for_each (coll.begin(), coll.end(),
              std::bind(std::mem_fn(&Person::print2),
                        std::placeholders::_1,
                        "Person: "));
```

Связывание с данными-членами

Привязку можно установить и к данным-членам. Рассмотрим следующий пример (пространство имен опущено):⁴

```
map<string,int> coll; // отображение целых чисел, ассоциированных со строками
...
// накапливаем все значения (вторые члены элементов)
int sum
= accumulate (coll.begin(), coll.end(),
              0,
              bind(plus<int>(),
                  _1,
                  bind(&map<string,int>::value_type::second,
                      _2)));
```

Здесь вызывается алгоритм `accumulate()`, использующий бинарный предикат для суммирования всех значений всех элементов (см. раздел 11.11.1). Однако, поскольку мы используем отображение, в котором элементами являются пары “ключ–значение”, для доступа к значению элемента мы с помощью привязки

```
bind(&map<string,int>::value_type::second, _2)
```

связываем второй аргумент каждого вызова предиката с его членом `second`.

⁴Этот пример основан на программе, позаимствованной из книги [Karlsson:Boost] с любезного разрешения автора.

Адаптеры `not1()` и `not2()`

Адаптеры `not1()` и `not2()` можно считать почти устаревшими⁵. Их единственное предназначение — отрицание стандартных функциональных объектов. Например:

```
std::sort (coll.begin(), coll.end(),
          std::not2(std::less<int>()));
```

Этот код выглядит более удобным, чем

```
std::sort (coll.begin(), coll.end(),
          std::bind(std::logical_not<bool>(),
                    std::bind(std::less<int>(), _1, _2)));
```

Однако для использования адаптеров `not1()` и `not2()` невозможно написать реальный сценарий, потому что здесь можно просто использовать другой стандартный функциональный объект.

```
std::sort (coll.begin(), coll.end(),
          std::greater_equal<int>());
```

Более важным является то, что вызов адаптера `not2()` для функционального объекта `less<>` в любом случае ошибочный. Вероятно, программист имел в виду изменение направления сортировки с возрастающей на убывающую. Но отрицание оператора `<` — это оператор `>=`, а не `>`. Фактически функциональный объект `greater_equal<>` приводит к непредсказуемым последствиям, потому что алгоритм `sort()` требует *строгого квазиупорядочения*, который обеспечивается операцией `<`, но не операцией `>=`, нарушающей требование антисимметричности (см. раздел 7.7). Таким образом, можно либо передать функциональный объект

```
greater<int>()
```

либо поменять порядок аргументов

```
bind(less<int>(), _2, _1)
```

Другие примеры использования адаптеров `not1()` и `not2()` с другими устаревшими функциональными адаптерами приведены в разделе 10.2.4.

10.2.3. Пользовательские функциональные объекты для функциональных адаптеров

Привязки можно использовать и с пользовательскими функциональными объектами. Следующий пример демонстрирует полное описание функционального объекта, возводящего первый аргумент в степень, равную значению второго аргумента.

```
// fo/fopow.hpp

#include <cmath>
template <typename T1, typename T2>
```

⁵ Действительно, они были близки к объявлению устаревшими уже в стандарте C++11 [N3198:DeprAdapt].


```
struct fopow
{
    T1 operator() (T1 base, T2 exp) const {
        return std::pow(base,exp);
    }
};
```

Отметим, что первый аргумент и возвращаемое значение имеют одинаковый тип T1, а показатель степени имеет другой тип, T2.

Следующая программа демонстрирует применение пользовательского функционального объекта `fopow<>()`. В частности, она использует функциональные адаптеры `fopow<>()` и `bind()`.

```
// fo/fopow1.cpp

#include <iostream>
#include <vector>
#include <algorithm>
#include <iterator>
#include <functional>
#include "fopow.hpp"
using namespace std;
using namespace std::placeholders;

int main()
{
    vector<int> coll = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };

    // выводим на экран число 3, возведенное в степени,
    // равные значениям всех элементов

    transform (coll.begin(), coll.end(),           // источник
               ostream_iterator<float>(cout, " "), // получатель
               bind(fopow<float,int>(),3,_1));      // операция
    cout << endl;

    // выводим на экран все элементы, возведенные в степень 3
    transform (coll.begin(), coll.end(),           // источник
               ostream_iterator<float>(cout, " "), // получатель
               bind(fopow<float,int>(),_1,3));      // операция
    cout << endl;
}
```

Программа выводит на экран следующие числа:

```
3 9 27 81 243 729 2187 6561 19683
1 8 27 64 125 216 343 512 729
```

Следует подчеркнуть, что функциональный объект `fopow<>()` реализован для типов `float` и `int`. Если число типа `int` используется и как основание, и как показатель степени, можно вызвать функцию `pow()` с двумя аргументами типа `int`, но этот вариант не является переносимым, поскольку по стандарту функция `pow()` перегружена для нескольких, но не для всех элементарных типов.

```
transform (coll.begin(), coll.end(),
          ostream_iterator<int>(cout, " "),
          bind1st(fopow<int,int>(),3)); // ОШИБКА: неоднозначность
```

Подробное описание этой проблемы приведено в разделе 17.3.

10.2.4. Устаревшие функциональные адаптеры

В табл. 10.3 перечислены классы стандартных функциональных адаптеров, существовавших в стандартной библиотеке C++ до появления стандарта C++11 и в настоящее время устаревших⁶. Ниже будут приведены примеры их использования.

Таблица 10.3. Устаревшие стандартные функциональные адаптеры

Выражение	Результат
<code>bind1st(<i>op</i>, <i>arg</i>)</code>	Вызывает операцию <code>op(arg, param)</code>
<code>bind2nd(<i>op</i>, <i>arg</i>)</code>	Вызывает операцию <code>op(param, arg)</code>
<code>ptr_fun(<i>op</i>)</code>	Вызывает операцию <code>*op(param)</code> или <code>*op(param1, param2)</code>
<code>mem_fun(<i>op</i>)</code>	Вызывает операцию <code>op()</code> как функцию-член объекта, на который ссылается указатель
<code>mem_fun_ref(<i>op</i>)</code>	Вызывает операцию <code>op()</code> как функцию-член объекта
<code>not1(<i>op</i>)</code>	Унарное отрицание: <code>!op(param)</code>
<code>not2(<i>op</i>)</code>	Бинарное отрицание: <code>!op(param1, param2)</code>

Эти адаптеры требуют определения некоторых типов в используемых функциональных объектах. Для определения этих типов в стандартной библиотеке C++ предусмотрены специальные базовые классы для функциональных адаптеров: `std::unary_function<>` и `std::binary_function<>`. В настоящее время эти классы также устарели.

Адаптеры `bind1st()` и `bind2nd()`, как и `bind()`, связывают параметр с фиксированными позициями. Например:

```
// найти первый элемент, превышающий 42
std::find_if (coll.begin(), coll.end(), // диапазон
             std::bind2nd(std::greater<int>(), 42)) // критерий
```

Однако адаптеры `bind1st()` и `bind2nd()` нельзя использовать для создания композиции привязок или для непосредственной передачи функций.

Адаптеры `not1()` и `not2()` “почти устарели”, потому что они полезны только при работе с другими устаревшими функциональными адаптерами. Например, алгоритм

```
std::find_if (coll.begin(), coll.end(),
             std::not1(std::bind2nd(std::modulus<int>(), 2)));
```

находит позицию первого четного целого числа (операция `%2` возвращает 0 для всех четных значений, а адаптер `not1()` отрицает 0, возвращая значение `true`). Однако это можно сделать более удобно с помощью новых привязок.

⁶ Кроме адаптеров `not1()` и `not2()`, которые не были официально объявлены устаревшими, в реальных программах встречаются и другие устаревшие функциональные адаптеры.

```
std::find_if (coll.begin(), coll.end(),
             std::bind(std::logical_not<bool>(),
                      std::bind(std::modulus<int>(),
                              std::placeholders::_1,
                              2)));
```

Еще лучше применить в этой ситуации лямбда-функцию.

```
std::find_if (coll.begin(), coll.end(),
             [](int elem){
                 return elem%2==0;
             });
```

Адаптер `ptr_fun()` позволяет вызывать обычные функции. Например, допустим, что у нас есть глобальная функция, проверяющая некое условие для каждого параметра.

```
bool check(int elem);
```

Для того чтобы найти первый элемент, не удовлетворяющий проверяемому условию, можно выполнить оператор

```
std::find_if (coll.begin(), coll.end(),          // диапазон
             std::not1(std::ptr_fun(check))); // критерий поиска
```

Вторая форма адаптера `ptr_fun()` используется тогда, когда у нас есть глобальная функция с двумя параметрами, которую мы хотим использовать как унарную функцию.

```
// найти первую непустую строку
std::find_if (coll.begin(), coll.end(),
             std::bind2nd(std::ptr_fun(std::strcmp), ""));
```

Здесь С-функция `strcmp()` используется для сравнения каждого элемента с пустой С-строкой. Если обе строки совпадают, то функция `strcmp()` возвращает 0, что эквивалентно значению `false`. Итак, этот вызов алгоритма `find_if()` возвращает позицию первого элемента, не являющегося пустой строкой.

Адаптеры `mem_fun()` и `mem_fun_ref()` позволяют определить функциональные объекты, вызывающие функции-члены⁷. Например:

```
class Person {
public:
    void print () const;
    ...
};

const std::vector<Person> coll;
...
// вызываем функцию-член print() из каждого объекта класса Person
std::for_each (coll.begin(), coll.end(),
              std::mem_fun_ref(&Person::print));
```

⁷Эти адаптеры функций-членов используют вспомогательные классы `mem_fun_t`, `mem_fun_ref_t`, `const_mem_fun_t`, `const_mem_fun_ref_t`, `mem_fun1_t`, `mem_fun1_ref_t`, `const_mem_fun1_t` и `const_mem_fun1_ref_t`.

Отметим, что функции-члены, вызываемые адаптерами `mem_fun_ref()` и `mem_fun()` и передаваемые как аргументы адаптерам `bind1st()` или `bind2nd()`, должны быть *константными*.

10.3. Использование лямбда-функций

Как указано в разделе 3.1.10, в стандарте C++11 появились лямбда-функции. Они представляют собой мощный и очень удобный механизм локальной функциональности, особенно полезный для уточнения деталей алгоритмов и функций-членов. Хотя лямбда-функции являются свойствами языка, их использование настолько важно для стандартной библиотеки C++, что я решил описать их здесь.

В разделе 6.9 отмечено, что лямбда-функции являются значительным улучшением языка C++ при работе с библиотекой STL, потому что теперь у нас есть интуитивный и понятный способ передачи индивидуального поведения алгоритмам и функциям-членам контейнеров. Если алгоритму требуется передать индивидуальное поведение, достаточно описать его как любую другую функцию именно в том месте, где он требуется.

Лучше всего продемонстрировать использование лямбда-функций на примере, сравнив его с соответствующим кодом, в котором не используются лямбда-функции. В следующих подразделах приведено несколько примеров функциональности, для достижения которой ранее использовались функциональные объекты и адаптеры, такие как `bind()`.

10.3.1. Лямбда-функции и адаптеры

Возьмем, например, файл `fo/bind1.cpp`, представленный в разделе 10.2.2. При использовании лямбда-функций соответствующий код выглядит следующим образом:

```
// fo/lambda1.cpp

#include <iostream>
int main()
{
    auto plus10 = [] (int i) {
        return i+10;
    };

    std::cout << "+10:      " << plus10(7) << std::endl;

    auto plus10times2 = [] (int i) {
        return (i+10)*2;
    };

    std::cout << "+10 *2: " << plus10times2(7) << std::endl;

    auto pow3 = [] (int i) {
        return i*i*i;
    };

    std::cout << "x*x*x: " << pow3(7) << std::endl;
```

```

auto inversDivide = [] (double d1, double d2) {
    return d2/d1;
};

std::cout << "invdiv: " << inversDivide(49,7) << std::endl;
}

```

Сравните это с объявлением функционального объекта. Объявление “добавить 10 и умножить на 2” с помощью привязок выглядит следующим образом:

```

auto plus10times2 = std::bind(std::multiplies<int>(),
    std::bind(std::plus<int>(),
        std::placeholders::_1,
        10),
    2);

```

Та же самая функциональная возможность, определенная с помощью лямбда-функции, выглядит так:

```

auto plus10times2 = [] (int i) {
    return (i+10)*2;
};

```

10.3.2. Лямбда-функции и функциональные объекты, имеющие состояние

Теперь заменим пользовательский функциональный объект лямбда-функцией. Рассмотрим пример с вычислением среднего значения элементов из раздела 10.1.3. Версия с лямбда-функциями приведена ниже.

```

// fo/lambda2.cpp

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main()
{
    vector<int> coll = { 1, 2, 3, 4, 5, 6, 7, 8 };

    // вычисляем и выводим среднее значение
    long sum = 0;
    for_each (coll.begin(), coll.end(), // диапазон
        [&sum] (int elem) {
            sum += elem;
        });
    double mv = static_cast<double>(sum)/static_cast<double>(coll.size());
    cout << "mean value: " << mv << endl;
}

```

Здесь вместо определения класса для передаваемого функционального объекта мы просто передаем требуемую функциональность. Однако результат вычислений хранится вне лямбда-функции в переменной `sum`, так что для вычисления среднего значения нам потребуется именно эта переменная.

При работе с функциональным объектом это состояние (переменная `sum`) полностью инкапсулируется, и мы можем предоставить для работы с этим состоянием дополнительные функции-члены (например, для получения среднего значения из переменной `sum`).

```
MeanValue mv = for_each (coll.begin(), coll.end(), // диапазон
                        MeanValue());           // операция
cout << "mean value: " << mv.value() << endl;
```

Так что с точки зрения вызовов пользовательский функциональный объект можно считать более компактным и менее подверженным ошибкам, чем лямбда-функции.

При работе с состоянием необходимо аккуратно использовать ключевое слово `mutable`. Рассмотрим пример из раздела 10.1.4, в котором критерий поиска третьего элемента задан функциональным объектом, имеющим состояние. Соответствующая версия, написанная с помощью лямбда-функций, строго говоря, должна передавать внутренний счетчик, выражающий ее состояние, по значению, потому что вне вызываемого алгоритма он не нужен. Используя ключевое слово `mutable`, можно обеспечить всем “вызовам функции” доступ для записи к этому состоянию.

```
// fo/lambda3.cpp

#include <iostream>
#include <list>
#include <algorithm>
#include "print.hpp"
using namespace std;

int main()
{
    list<int> coll = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
    PRINT_ELEMENTS(coll, "coll:      ");

    // удаляем третий элемент
    list<int>::iterator pos;
    int count=0;                               // счетчик вызовов
    pos = remove_if(coll.begin(), coll.end(), // диапазон
                   [count] (int) mutable { // критерий удаления
                       return ++count == 3;
                   });
    coll.erase(pos, coll.end());

    PRINT_ELEMENTS(coll, "3rd removed: ");
}
```

Однако, как указано в разделе 10.1.4, может возникнуть проблема с удалением третьего и шестого элементов. В итоге мы получим следующий результат:

```
coll: 1 2 3 4 5 6 7 8 9 10
3rd removed: 1 2 4 5 7 8 9 10
```

Как и ранее, причина ошибки заключается в том, что лямбда-объект копируется алгоритмом `remove_if()` во время его выполнения, так что существуют два лямбда-объекта, удаляющих третий элемент. Следовательно, состояние дублируется.

Если передать аргумент по ссылке и не использовать ключевое слово `mutable`, то все будет работать, как ожидается, потому что оба лямбда-объекта, используемые алгоритмом `remove_if()`, разделяют одно и то же состояние. Таким образом, код

```
int count=0;                                // счетчик вызовов
pos = remove_if(coll.begin(),coll.end(),    // диапазон
                [&count](int) {           // критерий удаления
                    return ++count == 3;
                });
```

выводит следующие результаты:

```
coll: 1 2 3 4 5 6 7 8 9 10
3rd removed: 1 2 4 5 6 7 8 9 10
```

10.3.3. Лямбда-функции, вызывающие глобальные функции и функции-члены

Разумеется, любая лямбда-функция может вызвать другую функцию, так что лямбда-версия программы `fo/compose3.cpp` из раздела 10.2.2 принимает следующий вид:

```
// fo/lambda4.cpp

#include <iostream>
#include <algorithm>
#include <locale>
#include <string>
using namespace std;

char myToupper (char c)
{
    std::locale loc;
    return std::use_facet<std::ctype<char> >(loc).toupper(c);
}

int main()
{
    string s("Internationalization");
    string sub("Nation");

    // поиск подстроки независимо от регистра
    string::iterator pos;
    pos = search (s.begin(),s.end(),          // строка, в которой идет поиск
                  sub.begin(),sub.end(),      // искомая подстрока
                  [] (char c1, char c2) {     // критерия сравнения
                      return myToupper(c1)==myToupper(c2);
                  });
    if (pos != s.end()) {
        cout << "\"" << sub << "\" is part of \"" << s << "\""
```

```
        << endl;
    }
}
```

Разумеется, точно так же можно вызывать функции-члены (сравните с разделом 10.2.2).

```
// fo/lambda5.cpp
```

```
#include <functional>
#include <algorithm>
#include <vector>
#include <iostream>
#include <string>
using namespace std;
using namespace std::placeholders;

class Person {
private:
    string name;
public:
    Person (const string& n) : name(n) {
    }

    void print () const {
        cout << name << endl;
    }

    void print2 (const string& prefix) const {
        cout << prefix << name << endl;
    }
    ...
};

int main()
{
    vector<Person> coll
        = { Person("Tick"), Person("Trick"), Person("Track") };

    // вызываем функцию-член print() каждого объекта класса Person
    for_each (coll.begin(), coll.end(),
        [] (const Person& p) {
            p.print();
        });
    cout << endl;

    // вызываем функцию-член print2() с дополнительным аргументом
    // из каждого объекта класса Person
    for_each (coll.begin(), coll.end(),
        [] (const Person& p) {
            p.print2("Person: ");
        });
}
```


10.3.4. Лямбда-функции как функции-хеширования, критерий сортировки и критерий эквивалентности

Как указывалось ранее, лямбда-функции можно использовать в качестве хеш-функций, а также критериев порядка и сортировки. Рассмотрим пример:

```
class Person {
    ...
};

auto hash = [] (const Person& p) {
    ...
};

auto eq = [] (const Person& p1, const Person& p2) {
    ...
};

// создаем неупорядоченное множество с пользовательским поведением
unordered_set<Person, decltype(hash), decltype(eq)> pset(10, hash, eq);
```

И снова подчеркнем, что мы должны использовать ключевое слово `decltype` для передачи типа лямбда-функций в контейнер `unordered_set`, потому что он создает их копии. Кроме того, конструктору необходимо передать хеш-функцию и критерий эквивалентности, потому что в противном случае конструктор вызывает конструктор по умолчанию для хеш-функции и критерия эквивалентности, которые для лямбда-функций не определены.

Из-за этих неудобств определение класса для функциональных объектов можно считать более читабельным и даже удобным. Итак, если в программе используется состояние, лямбда-функции не всегда лучше.

Полный пример использования лямбда-функций для определения хеш-функции и критерия эквивалентности для неупорядоченных контейнеров см. в разделе 7.9.7.

